

Lab assignment 1: Object recognition using deep networks

Challenge-based exercise

By Ben Harvey

This **Challenge-based exercise** focusses on your extending and assessing your understanding of the content you have covered in Learning-based Exercise One. It forms 10% of your course grade, though it requires less work.

- Here you will run a more complex model that recognises objects in natural images.
- The principles are similar and your answers here benefit from what you learned in Learning-based Exercise One. So, it is best to start working on this after completing Learning-based Exercise One at least, considering what you learned in class.
- Notably, the resulting model can take a couple of hours to train, depending on your hardware. Long training times are very common in deep networks. During this time, you may want something else to do. The first question to research and answer is therefore placed at that stage.
- You will not get input from your teachers, except to clarify the questions. This exercise can be complete during class or outside of class time. Notably, you can get plenty of input from your teachers to help you improve Learning-based Exercise TWO, so prioritise that exercise in class time.
- You should **NOT** work with another pair: each pair should work alone. Very similar answers in different pairs may be seen to reflect plagiarism.
- Each pair should submit a separate answers document specifically for the challenge-based exercise, at the separate submission link on Brightspace.

Let's begin:

Identifying objects from images with a high variation in object position, size and viewing angle is a particularly difficult problem, and a major application of artificial deep learning networks. Here we will build a network to do this. This network has more layers and is more computationally-intensive than the learning-based exercises, even using the low-resolution images we will use here.

The CIFAR-10 dataset contains colour images of objects, each 32x32x3 pixels (for the three colour channels). These have 10 categories (or classes) of object (airplane, automobile, bird, cat, deer, dog, frog, horse, ship & truck) with 5,000 images in each, making a total of 50,000 images in the training set (x_{train} , y_{train}), randomly ordered with numerical labels for each (1=airplane, 2=automobile etc.). The test set (x_{test} , y_{test}) contains 10,000 images ordered by their label.

First, download and load the data set using:

```
(x_train, y_train), (x_test, y_test) =  
keras.datasets.cifar10.load_data()
```

Prepare the test and training images by dividing their values by 255, storing the result in variables x_{train} and x_{test} . The shapes of the image matrices are already correct for input into Keras. Convert the training and test labels to categorical

variables, as for the handwritten digits in Learning-based Exercise One, storing the result in variables `y_train` and `y_test`.

Define the model using the convolutional network with dropout (from Learning-based Exercise One's 'Deep convolutional networks' topic (Questions 6-7)) as a template.

-For the first convolutional layer, add the arguments:

`input_shape = (32, 32, 3), padding = "same"`
to the layer `conv_2d` call.

- In the second convolutional layer, use 32 filters instead of 64 to reduce computational load.

-After max pooling and dropout layers, repeat these layers again (add `conv`, `conv`, `pool`, `dropout`, after the existing `conv`, `conv`, `pool`, `dropout`). There is no need to define `input_shape` here.

-Flatten the result and link it to a larger fully-connected layer than before, using 512 units instead of 128, with dropout as before.

-Link this to a 10-unit output layer as before.

In compiling the model, we will use a specialised optimizer module:

```
optimizer =  
keras.optimizers.RMSprop(learning_rate=float(0.0001),  
weight_decay=1e-6)
```

Next we will fit the model, which **will take at least several minutes but up to couple of hours(!)** (Google Colab will be much faster). During fitting, you can research and complete Questions 1, below, so you don't need to wait. You may choose to run this overnight, particularly if each training epoch lasts more than 10 minutes.

In the model fit command, set the batch size to 32 and the number of epochs to 20. Rather than splitting the training data to give a validation set, we will pass in the test set as follows:

```
validation_data=(x_test, y_test),
```

Finally, set:

```
shuffle = True
```

to run through the images in a different order of batches each epoch.

Execute this model fit command.

While you wait...

Research and describe the principles of overfitting. Describe how AND WHY dropout can reduce this, **citing your sources**. (Question 1, 5 points). You will learn more here by using web search tools (like Google) to find and integrate information from several sources, rather than using automatic text generators like ChatGPT. If you do use automatic text generators, explain in your answer how you used them in your research. Attribute any direct quotes to the text generators or web page that you used. Even when you are not quoting directly, please reference your answer to cite the sources of information that you used, wherever they came from.

Students typically take at least 15 minutes to do this well, though automatic text generators will do it less well in 2-4 seconds and teach you much less.

After your fitting is finished...

Plot the training history (as in Learning-based Exercise One). Include this in your answer document. (Question 2, 2 points)

How much longer does it take to run each training epoch for this model, compared to the most similar convolutional model for digit recognition (Learning-based Exercise One, Question 7?) (This should be clear, even in Google Colab). (Question 3, 1 point)

Give at least two factors that may contribute to this difference, explaining each. (Question 4, 4 points)

Describe in detail how the training histories of these two models differ, and why. Note the details, as you did in the learning-based assignment. (Question 5, 5 points). So, try to make at least 5 distinct points here. Note that this question is about convolutional models applied to different tasks.

The simpler, non-convolutional models from Learning-based Exercise One (Questions 2-5) would perform very poorly on this object recognition task, while they do fairly well on number recognition. What differences between these tasks could underlie this difference in performance models? (Question 6, 3 points). Note that this question is about non-convolutional (fully connected) models.